

Numerical Computing II — Assignment-I

Name: Kunal Dnyaneshwar Kamod | Roll No: MS2513
Class: M.Sc. Scientific Computing Modeling and Simulation

Paper I Choose

For this assignment I picked a paper on **credit card fraud detection**, because it falls under the Fraud Detection / **Machine Learning** with **Numerical Optimization** category, and it uses actual numerical methods instead of just being a general survey.

Title:

An Intelligent Approach to Credit Card Fraud Detection Using an Optimized Light Gradient Boosting Machine

Authors:

Altyeb Altaher Taha and Sharaf Jameel Malebary

Published in:

IEEE Access, Volume 8, pages 25579–25587, year 2020 (open access, peer-reviewed journal).

Link:

<https://ieeexplore.ieee.org/document/8979331>

Why I think this fits our course:

The paper builds a fraud-detection model called **OLightGBM**. What made me pick it is that it actually uses two layers of numerical optimization — one to train the model (**gradient boosting**) and one to tune the model's settings (**Bayesian optimization**). Both of these are iterative, number-crunching processes, not closed-form formulas, which is exactly the kind of thing we've been studying in class.

My Analysis

1. What problem are they solving?

Basically, they're trying to figure out whether a credit card transaction is fraud or genuine. They tested their model on two real datasets — a **European cardholders** dataset and something called the **UCSD-FICO** dataset. Both datasets have a big problem: fraud cases are very rare compared to normal transactions, so the data is heavily imbalanced.

2. Why can't this be solved with a normal formula?

There's no simple equation connecting things like transaction amount, time, or account behavior to a "fraud" label. The relationship is messy, non-linear, and keeps changing because fraudsters keep changing their tactics too. On top of that, the model itself (**LightGBM**) has a bunch of settings that control how it learns, and there's no formula to tell you the best combination — you can only find out by testing, which is why they had to use a numerical search method.

3. What kind of numerical computation is actually happening?

- **Training the model:** **LightGBM** builds one **decision tree**, checks the **errors**, builds another tree to fix those errors, and keeps repeating — this is basically **gradient descent** done in steps.
- **Tuning the model:** they use **Bayesian Optimization**, which builds a rough "guess" of how good different settings will be, and keeps updating that guess after every test, instead of trying every possible combination.
- **Checking the model:** they used **5-fold cross-validation**, meaning they split the data into 5 parts and tested it 5 different times, then averaged the results.
- They also calculated **accuracy, precision, recall, F1-score, and AUC** (area under the ROC curve), which all involve **numerical calculation**.

4. Numerical Computing topics I can connect to this paper

- Iterative optimization (gradient descent style training in boosting).
- Optimization without derivatives (Bayesian optimization).
- Numerical approximation (histogram binning used inside LightGBM to speed things up).
- Statistics and error estimation (**cross-validation**, averaging results).
- Numerical integration (**calculating area under the ROC curve for AUC**).

5. What software did they use?

From what I could find, the main algorithm is LightGBM (originally introduced by **Ke et al. in 2017**), and it's tuned using Bayesian optimization. I couldn't confirm the exact programming language or optimization library the authors used in their actual experiments, so I'm not going to guess and claim it as fact — I'm just noting that **LightGBM** is commonly used through Python in general.

6. What surprised me

Honestly what caught my attention was the **F1-score**. The accuracy was **98.40%** and **precision** was **97.34%**, which sounds almost perfect, but the **F1-score** was only **56.95%**. At first I thought that was a mistake, but then I realized it's because of the **imbalance** — since fraud cases are so rare, a model can look amazing on **accuracy** while still missing a good chunk of actual fraud cases. That's something I hadn't really thought about before this assignment.

Summary

Taha and Malebary (IEEE Access, 2020) built a fraud detection model called OLightGBM, combining LightGBM (a fast, tree-based gradient boosting algorithm) with Bayesian optimization to automatically tune its settings instead of tuning them manually. They tested it on two real, imbalanced credit card datasets using **5-fold cross-validation** and got **98.40% accuracy, 92.88% AUC, 97.34% precision, and 56.95% F1-score**, beating the methods they compared it with. What I liked about this paper for our course is that it shows two layers of numerical optimization working together — one training the actual model step by step, and another one tuning the model's configuration — with no shortcut formula involved anywhere. It's a good, real example of how something as everyday as swiping a credit card is actually being decided by numerical algorithms running quietly in the background.

Key Points I Took Away

- Main algorithms: **LightGBM** (gradient boosted **decision trees**) and **Bayesian hyperparameter optimization**.
- Good things: automatic tuning saves time, model is fast on large data, **high accuracy** and **precision** on two real datasets.
- Weak points: **F1-score is much lower**, meaning **imbalance is still an issue**; tree models like this are hard to interpret compared to simple formulas.
- Not confirmed anywhere I found: their exact software setup and their own stated future work — so I haven't written those in as if they were facts.

Reference

A. A. Taha and S. J. Malebary, "An Intelligent Approach to Credit Card Fraud Detection Using an Optimized Light Gradient Boosting Machine," IEEE Access, vol. 8, pp. 25579–25587, 2020, doi: 10.1109/ACCESS.2020.2971354.